

SEGUNDO PARCIAL - TECNICAS ALGORITMICAS

Pérez Barahona Pedro Luis

3 de abril de 2025

En este documento se presenta el desarrollo de una solución algorítmica utilizando el algoritmo de Dijkstra, aplicado a grafos dirigidos y ponderados. Este trabajo forma parte del segundo parcial de la asignatura Técnicas Algorítmicas, y tiene como propósito fundamental implementar un sistema capaz de determinar la ruta más corta entre dos vértices de un grafo, considerando el peso de sus aristas.

Se ofrece al usuario la posibilidad de definir la cantidad de nodos del grafo, ingresar manualmente los pesos de cada conexión o generarlos aleatoriamente, y posteriormente visualizar el grafo y la tabla de costos mínimos. Además, se permite seleccionar un nodo de origen y uno de destino para obtener el camino óptimo, su costo total y una visualización gráfica del recorrido más eficiente.

Este proyecto está basado en los lineamientos establecidos en la Guía 10 de la asignatura, respetando las condiciones de que el grafo sea dirigido y no contenga aristas con pesos negativos. La implementación se realizó en Google Colab con Python, y la documentación del proceso se estructura en este reporte en formato L^AT_EX.

1. Introducción

En el presente trabajo se desarrolla un proyecto práctico basado en el uso del algoritmo de Dijkstra para la resolución de problemas relacionados con grafos dirigidos y ponderados. Esta actividad se enmarca dentro del segundo parcial de la asignatura Técnicas Algorítmicas, y sigue los lineamientos propuestos en la Guía 10.

El objetivo principal es calcular el camino más corto entre un vértice origen y un vértice destino dentro de un grafo, considerando el peso o costo de cada arista. Para ello, se diseñó una libreta interactiva en Google Colab utilizando Python, que permite al usuario definir la cantidad de nodos del grafo, ingresar manualmente los pesos o generarlos aleatoriamente, y visualizar la estructura del grafo y su matriz de adyacencia.

Posteriormente, el sistema permite seleccionar un nodo inicial y uno final, aplicar el algoritmo de Dijkstra para determinar la ruta más eficiente, y presentar los resultados tanto en forma tabular como gráfica. La visualización incluye el grafo con el camino óptimo resaltado, así como una tabla con los costos mínimos hacia cada nodo y un resumen textual del recorrido más corto encontrado.

Este proyecto pone en práctica los fundamentos del diseño de algoritmos, la manipulación de estructuras de datos como matrices y grafos, y el desarrollo de soluciones computacionales eficientes orientadas a la toma de decisiones. El algoritmo de Dijkstra es una herramienta clásica en este contexto, destacando por su enfoque voraz y su aplicabilidad a problemas reales como la optimización de rutas, planificación de redes y logística.

2. Conceptos clave

Para abordar el problema de encontrar rutas óptimas en un grafo, es necesario comprender ciertos conceptos fundamentales. Un **grafo** es una estructura compuesta por un conjunto de nodos o vértices conectados entre sí mediante enlaces llamados *aristas*. Cuando estas aristas tienen una dirección específica, se habla de un **grafo dirigido**. Si además cada arista tiene un valor numérico asociado que representa un costo, distancia o peso, se denomina **grafo ponderado**.

En este contexto, la **matriz de adyacencia** es una representación matricial del grafo donde cada celda $M[i][j]$ indica el peso de la conexión del nodo i al nodo j , o un valor especial (como -1 o ∞) si no existe tal conexión. Estos conceptos permiten estructurar de manera computacional problemas reales

como la planificación de rutas, redes de transporte o flujos de comunicación, que pueden resolverse mediante algoritmos de caminos más cortos.

Algoritmo de Dijkstra El **algoritmo de Dijkstra** es una técnica eficiente para encontrar el camino de costo mínimo desde un nodo origen hacia todos los demás nodos de un grafo dirigido y ponderado, siempre que las aristas tengan pesos no negativos. Fue propuesto por Edsger W. Dijkstra en 1959 y es considerado un algoritmo voraz, ya que en cada iteración elige la opción óptima local esperando alcanzar la solución global.

El algoritmo parte desde el nodo inicial y explora sucesivamente los caminos posibles, actualizando las distancias mínimas conocidas hacia cada nodo, y marcando aquellos ya evaluados. Utiliza una estructura de datos para llevar un registro de los nodos visitados y sus respectivas distancias, garantizando que al finalizar el proceso, la ruta más corta hacia cada nodo ha sido correctamente calculada. Dijkstra es ampliamente utilizado en sistemas de navegación, redes de computadoras, inteligencia artificial y muchas otras áreas donde se requiere optimización de rutas.

3. Marco Teórico

Conceptos clave Para abordar el problema de encontrar rutas óptimas en un grafo, es necesario comprender ciertos conceptos fundamentales. Un **grafo** es una estructura compuesta por un conjunto de nodos o vértices conectados entre sí mediante enlaces llamados *aristas*. Cuando estas aristas tienen una dirección específica, se habla de un **grafo dirigido**. Si además cada arista tiene un valor numérico asociado que representa un costo, distancia o peso, se denomina **grafo ponderado**.

En este contexto, la **matriz de adyacencia** es una representación matricial del grafo donde cada celda $M[i][j]$ indica el peso de la conexión del nodo i al nodo j , o un valor especial (como -1 o ∞) si no existe tal conexión. Estos conceptos permiten estructurar de manera computacional problemas reales como la planificación de rutas, redes de transporte o flujos de comunicación, que pueden resolverse mediante algoritmos de caminos más cortos.

Algoritmo de Dijkstra El **algoritmo de Dijkstra** es una técnica eficiente para encontrar el camino de costo mínimo desde un nodo origen hacia todos los demás nodos de un grafo dirigido y ponderado, siempre que las aristas tengan pesos no negativos. Fue propuesto por Edsger W. Dijkstra en 1959 y es considerado un algoritmo voraz, ya que en cada iteración elige la opción óptima local esperando alcanzar la solución global.

El algoritmo parte desde el nodo inicial y explora sucesivamente los caminos posibles, actualizando las distancias mínimas conocidas hacia cada nodo, y marcando aquellos ya evaluados. Utiliza una estructura de datos para llevar un registro de los nodos visitados y sus respectivas distancias, garantizando que al finalizar el proceso, la ruta más corta hacia cada nodo ha sido correctamente calculada. Dijkstra es ampliamente utilizado en sistemas de navegación, redes de computadoras, inteligencia artificial y muchas otras áreas donde se requiere optimización de rutas.

4. Desarrollo

Durante el desarrollo del proyecto, se diseñó una interfaz interactiva que permite al usuario:

- Definir el número de nodos del grafo.
- Elegir entre ingresar los pesos de forma manual o aleatoria.
- Visualizar la matriz de adyacencia y el grafo generado.
- Seleccionar un nodo inicial y un nodo final.
- Ejecutar el algoritmo de Dijkstra.
- Obtener el camino más corto, su costo total, una tabla de resultados y un gráfico con el recorrido resaltado.

También se incorporó una salida en formato imagen para la tabla final, junto a un resumen textual del recorrido más eficiente. Esta funcionalidad facilita la presentación y documentación de los resultados, especialmente en entornos académicos.

5. Implementación en el entorno Python Jupyter Notebook (Google Colab)

La elección del entorno **Google Colab** se basó en su facilidad de uso, compatibilidad con Python, y acceso gratuito desde cualquier navegador. Este entorno ofrece todas las ventajas de Jupyter Notebook pero con la potencia de la nube, lo que permite desarrollar, ejecutar y visualizar algoritmos de forma interactiva sin necesidad de instalar software adicional.

Además, Colab permite integrar gráficos, celdas interactivas, cargar archivos, generar imágenes y exportar resultados, todo en un solo documento ejecutable. Su enfoque visual e inmediato lo convierte en una herramienta ideal para enseñar y experimentar con algoritmos como el de Dijkstra, facilitando tanto el desarrollo como la comprensión del problema y sus soluciones.

5.1. Visualización gráfica del número de nodos

El siguiente fragmento de código permite al usuario ingresar cuántos nodos tendrá el grafo, mostrando luego una imagen simple con el mensaje visual “Tu grafo tendrá n nodos”. Esta interfaz gráfica mejora la interactividad con el usuario, generando una respuesta visual mediante `matplotlib`.

```
1 import matplotlib.pyplot as plt
2
3 # Pedir n mero de nodos
4 n = int(input(" Cuntos      nodos tendr  tu grafo?: "))
5
6 # Mostrar mensaje en imagen
7 fig, ax = plt.subplots()
8 ax.text(0.5, 0.5, f"Tu grafo tendr  {n} nodos", fontsize=20, ha='center', va='center'
9         )
10 ax.axis('off')
11 plt.show()
```

Listing 1: Visualización de la cantidad de nodos del grafo

5.2. Ingreso de los pesos del grafo

En esta sección se permite al usuario decidir cómo asignar los pesos a las aristas del grafo. Puede optar por generar los valores aleatoriamente con una probabilidad del 50 % entre cada par de nodos, o bien ingresar cada peso de manera manual. Se asegura que los pesos sean no negativos, cumpliendo con la restricción del algoritmo de Dijkstra. Los valores no conectados se representan con `-1`, y al final se muestra la matriz generada en forma de tabla.

```
1 # Mostrar opciones
2 print(" Cmo  deseas ingresar los pesos del grafo?")
3 print("1. Generar pesos aleatorios")
4 print("2. Ingresar los pesos manualmente")
5
6 opcion = int(input("Selecciona una opci n (1 o 2): "))
7 matriz = np.full((n, n), -1) # Inicializar con -1 como indica la gu a
8
9 if opcion == 1:
10     for i in range(n):
11         for j in range(n):
12             if i != j:
13                 # 50% de probabilidad de que exista una arista
14                 if np.random.rand() > 0.5:
15                     matriz[i][j] = np.random.randint(1, 20)
16 elif opcion == 2:
17     for i in range(n):
18         for j in range(n):
19             if i != j:
```

```

20         peso = input(f"Ingresa el peso de la arista de {i} a {j} (o presiona
21             Enter si no hay conexi n): ")
22         if peso.strip() != "":
23             peso = int(peso)
24             if peso >= 0:
25                 matriz[i][j] = peso
26             else:
27                 print("          El peso debe ser no negativo. Se omitir esta
28                     conexi n.")
29         else:
30             print("          Opci n inv lida. Ejecuta la celda nuevamente.")
31 # Convertir a DataFrame para visualizaci n
32 df = pd.DataFrame(matriz, columns=[f"Node {i}" for i in range(n)],
33                     index=[f"Node {i}" for i in range(n)])
34 print("          Matriz de pesos del grafo (con -1 donde no hay conexi n):")
35 df

```

Listing 2: Ingreso de pesos: aleatorio o manual

5.3. Generaci3n y visualizaci3n del grafo dirigido

En esta etapa se construye un grafo dirigido a partir de la matriz de adyacencia definida previamente. Se utiliza la librería `networkx` para crear la estructura de nodos y aristas, y `matplotlib` para la visualizaci3n gráfica. Cada arista del grafo contiene un peso representado como una etiqueta en color rojo. Los nodos se distribuyen automáticamente mediante un layout de resorte (`spring_layout`).

```

1 # Crear grafo dirigido
2 G = nx.DiGraph()
3
4 # Agregar nodos
5 G.add_nodes_from(range(n))
6
7 # Agregar aristas con pesos
8 for i in range(n):
9     for j in range(n):
10         if matriz[i][j] != -1:
11             G.add_edge(i, j, weight=matriz[i][j])
12
13 # Posiciones autom ticas para los nodos
14 pos = nx.spring_layout(G, seed=42) # Puedes cambiar el layout si deseas
15
16 # Dibujar nodos y aristas
17 plt.figure(figsize=(10, 6))
18 nx.draw(G, pos, with_labels=True, node_color='lightblue',
19         node_size=1000, arrows=True, font_size=12)
20
21 # Dibujar etiquetas de pesos
22 labels = nx.get_edge_attributes(G, 'weight')
23 nx.draw_networkx_edge_labels(G, pos, edge_labels=labels, font_color='red')
24 plt.title("          Grafo dirigido con pesos")
25 plt.show()

```

Listing 3: Visualizaci3n del grafo dirigido con pesos

5.4. Implementaci3n del algoritmo de Dijkstra

A continuaci3n se presenta la implementaci3n del algoritmo de Dijkstra, el cual permite calcular la distancia m3nima desde un nodo de origen hacia todos los dem3s nodos del grafo. El algoritmo selecciona iterativamente el nodo con menor distancia acumulada que a3n no ha sido visitado, y actualiza las distancias de sus vecinos si encuentra un camino m3s corto. Luego de ejecutar la funci3n, se imprime el costo m3nimo desde el nodo seleccionado a cada uno de los dem3s.

```

1 def dijkstra(matriz, origen):
2     n = len(matriz)
3     visitados = [False] * n

```

```

4 distancia = [np.inf] * n
5 distancia[origen] = 0
6
7 for _ in range(n):
8     # Seleccionar el nodo no visitado con menor distancia
9     min_dist = np.inf
10    nodo_actual = -1
11    for i in range(n):
12        if not visitados[i] and distancia[i] < min_dist:
13            min_dist = distancia[i]
14            nodo_actual = i
15
16    if nodo_actual == -1:
17        break
18
19    visitados[nodo_actual] = True
20
21    # Actualizar las distancias de los vecinos del nodo actual
22    for vecino in range(n):
23        peso = matriz[nodo_actual][vecino]
24        if peso != -1 and not visitados[vecino]:
25            nueva_distancia = distancia[nodo_actual] + peso
26            if nueva_distancia < distancia[vecino]:
27                distancia[vecino] = nueva_distancia
28
29    return distancia
30
31 # Pedir nodo de origen
32 origen = int(input(f"Selecciona el nodo de origen (0 a {n-1}): "))
33
34 # Ejecutar algoritmo
35 distancias = dijkstra(matriz, origen)
36
37 # Mostrar resultados
38 print(f"\n      Distancias mínimas desde el nodo {origen}:\n")
39 for i, d in enumerate(distancias):
40     if d == np.inf:
41         print(f"      No hay camino de {origen} a {i}")
42     else:
43         print(f"      Distancia de {origen} a {i}: {d}")

```

Listing 4: Cálculo de distancias mínimas usando Dijkstra

5.5. Cálculo del camino más corto entre dos nodos

Esta sección permite al usuario seleccionar dos nodos específicos: uno de origen y uno de destino. Se ejecuta una versión modificada del algoritmo de Dijkstra que, además de calcular la distancia mínima, guarda el recorrido óptimo seguido. Luego, se reconstruye el camino usando la lista de predecesores (*anterior*) y se imprime la ruta junto con el costo total.

```

1 def dijkstra_camino(matriz, origen):
2     n = len(matriz)
3     distancia = [np.inf] * n
4     visitados = [False] * n
5     anterior = [None] * n
6     distancia[origen] = 0
7
8     for _ in range(n):
9         min_dist = np.inf
10        nodo_actual = -1
11        for i in range(n):
12            if not visitados[i] and distancia[i] < min_dist:
13                min_dist = distancia[i]
14                nodo_actual = i
15
16        if nodo_actual == -1:
17            break
18
19        visitados[nodo_actual] = True

```

```

20
21     for vecino in range(n):
22         peso = matriz[nodo_actual][vecino]
23         if peso != -1 and not visitados[vecino]:
24             nueva_dist = distancia[nodo_actual] + peso
25             if nueva_dist < distancia[vecino]:
26                 distancia[vecino] = nueva_dist
27                 anterior[vecino] = nodo_actual
28
29     return distancia, anterior
30
31 # Mostrar nodos disponibles
32 print(f"Nodos disponibles: {[i for i in range(n)]}")
33 origen = int(input("Selecciona el nodo inicial: "))
34
35 # Mostrar nodos restantes
36 restantes = [i for i in range(n) if i != origen]
37 print(f"Nodos destino disponibles: {restantes}")
38 destino = int(input("Selecciona el nodo final: "))
39
40 # Ejecutar algoritmo y reconstruir camino
41 distancias, anteriores = dijkstra_camino(matriz, origen)
42
43 # Reconstruir el camino
44 camino = []
45 nodo = destino
46 while nodo is not None:
47     camino.insert(0, nodo)
48     nodo = anteriores[nodo]
49
50 # Mostrar resultados
51 if distancias[destino] == np.inf:
52     print(f"          No hay camino de {origen} a {destino}")
53 else:
54     print(f"\n Ruta m s corta de {origen} a {destino}: {' '.join(map(str,
55         camino))}")
56     print(f"          Costo m nimo: {distancias[destino]}")

```

Listing 5: Cálculo y reconstrucción del camino óptimo entre dos nodos

5.6. Visualización del camino más corto en el grafo

Una vez calculado el camino óptimo entre el nodo origen y el nodo destino, se realiza una visualización diferenciada del grafo. Las aristas que forman parte del camino más corto se dibujan en color rojo, y los nodos que lo componen se resaltan en verde. El resto del grafo se representa en gris para contraste. Esta visualización permite al usuario interpretar rápidamente la ruta seleccionada por el algoritmo de Dijkstra.

```

1 # Crear grafo dirigido
2 G = nx.DiGraph()
3
4 # Agregar nodos
5 G.add_nodes_from(range(n))
6
7 # Agregar aristas con pesos
8 for i in range(n):
9     for j in range(n):
10         if matriz[i][j] != -1:
11             G.add_edge(i, j, weight=matriz[i][j])
12
13 # Posición de nodos (puedes cambiar por otro layout si deseas)
14 pos = nx.spring_layout(G, seed=42)
15
16 # Extraer aristas del camino m s corto
17 camino_aristas = [(camino[i], camino[i+1]) for i in range(len(camino)-1)]
18
19 # Dibujar todos los nodos y aristas en gris
20 nx.draw_networkx_edges(G, pos, edge_color='lightgray')
21 nx.draw_networkx_nodes(G, pos, node_color='lightgray', node_size=800)

```

```

22 nx.draw_networkx_labels(G, pos, font_size=12)
23
24 # Dibujar aristas del camino m s corto en rojo
25 nx.draw_networkx_edges(G, pos, edgelist=camino_aristas, edge_color='red', width=3)
26
27 # Dibujar nodos del camino m s corto en verde
28 nx.draw_networkx_nodes(G, pos, nodelist=camino, node_color='lightgreen', node_size
    =900)
29
30 # Mostrar etiquetas de pesos
31 edge_labels = nx.get_edge_attributes(G, 'weight')
32 nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='black')
33
34 plt.title(f"          Camino m s corto de {origen} a {destino} (Costo: {distancias[
    destino]})", fontsize=14)
35 plt.axis('off')
36 plt.show()

```

Listing 6: Grafo con el camino más corto resaltado

5.7. Tabla visual del recorrido y resumen del resultado

Finalmente, se genera una tabla visual con colores que resume los costos mínimos desde el nodo de origen hacia los demás nodos del grafo. Esta tabla destaca con colores el nodo origen, el nodo destino y los nodos que forman parte del camino más corto. Además, se imprime un resumen textual que explica el recorrido tomado y el costo total de dicha ruta, indicando cada transición con su respectivo peso.

```

1 import matplotlib.pyplot as plt
2
3 # Preparar datos para mostrar en imagen
4 column_labels = ["Nodo", "Costo Final", "Costo Temporal"]
5 cell_text = []
6
7 for i in range(n):
8     final = distancias[i] if distancias[i] != np.inf else "-"
9     temporal = final
10    cell_text.append([f"Nodo {i}", final, temporal])
11
12 # Colores para resaltar
13 row_colors = []
14 for i in range(n):
15     if i == origen:
16         row_colors.append("lightgreen") # Nodo origen
17     elif i == destino:
18         row_colors.append("lightblue") # Nodo destino
19     elif i in camino:
20         row_colors.append("mistyrose") # Parte del camino
21     else:
22         row_colors.append("white")
23
24 # Crear figura con tabla
25 fig, ax = plt.subplots(figsize=(7, n * 0.5 + 2))
26 ax.axis("off")
27 tabla_img = ax.table(cellText=cell_text,
28                      colLabels=column_labels,
29                      cellLoc='center',
30                      loc='center',
31                      rowColours=row_colors)
32 tabla_img.auto_set_font_size(False)
33 tabla_img.set_fontsize(12)
34 tabla_img.scale(1, 1.2)
35
36 # T tulo
37 plt.title(f"          Camino m s corto de Nodo {origen} a Nodo {destino} - Costo: {
    distancias[destino]})", fontsize=14)
38 plt.show()
39
40 # Explicaci n tipo reporte
41 if distancias[destino] == np.inf:

```

```

42     print(f"          No hay camino posible desde el nodo {origen} hasta el nodo {destino
43           }.")
44 else:
45     detalle_ruta = []
46     for i in range(len(camino)-1):
47         u = camino[i]
48         v = camino[i+1]
49         peso = matriz[u][v]
50         detalle_ruta.append(f"({u}    {v})({peso})")
51
52     ruta_str = "    ".join(detalle_ruta)
53     print(f"          El nodo v rtice {origen} tiene como camino m s corto hacia el
54           nodo v rtice {destino} un costo total de {distancias[destino]}.")
55     print(f"          Esto se debe a que tom  el siguiente camino seg n los pesos de
56           las aristas:")
57     print(f"          {ruta_str}")

```

Listing 7: Tabla de costos y resumen explicativo del camino

5.8. Tabla visual del recorrido y resumen del resultado

Finalmente, se genera una tabla visual con colores que resume los costos mínimos desde el nodo de origen hacia los demás nodos del grafo. Esta tabla destaca con colores el nodo origen, el nodo destino y los nodos que forman parte del camino más corto. Además, se imprime un resumen textual que explica el recorrido tomado y el costo total de dicha ruta, indicando cada transición con su respectivo peso.

```

1 import matplotlib.pyplot as plt
2
3 # Preparar datos para mostrar en imagen
4 column_labels = ["Nodo", "Costo Final", "Costo Temporal"]
5 cell_text = []
6
7 for i in range(n):
8     final = distancias[i] if distancias[i] != np.inf else "--"
9     temporal = final
10    cell_text.append([f"Nodo {i}", final, temporal])
11
12 # Colores para resaltar
13 row_colors = []
14 for i in range(n):
15     if i == origen:
16         row_colors.append("lightgreen") # Nodo origen
17     elif i == destino:
18         row_colors.append("lightblue") # Nodo destino
19     elif i in camino:
20         row_colors.append("mistyrose") # Parte del camino
21     else:
22         row_colors.append("white")
23
24 # Crear figura con tabla
25 fig, ax = plt.subplots(figsize=(7, n * 0.5 + 2))
26 ax.axis("off")
27 tabla_img = ax.table(cellText=cell_text,
28                      colLabels=column_labels,
29                      cellLoc='center',
30                      loc='center',
31                      rowColours=row_colors)
32 tabla_img.auto_set_font_size(False)
33 tabla_img.set_fontsize(12)
34 tabla_img.scale(1, 1.2)
35
36 # T tulo
37 plt.title(f"          Camino m s corto de Nodo {origen} a Nodo {destino} - Costo: {
38           distancias[destino]}", fontsize=14)
39 plt.show()
40
41 # Explicaci n tipo reporte
42 if distancias[destino] == np.inf:
43     print(f"          No hay camino posible desde el nodo {origen} hasta el nodo {destino
44           }.")

```



```

43 else:
44     detalle_ruta = []
45     for i in range(len(camino)-1):
46         u = camino[i]
47         v = camino[i+1]
48         peso = matriz[u][v]
49         detalle_ruta.append(f"({u}    {v})({peso})")
50
51     ruta_str = "    ".join(detalle_ruta)
52     print(f"        El nodo v rtice {origen} tiene como camino m s corto hacia el
53           nodo v rtice {destino} un costo total de {distancias[destino]}.")
54     print(f"        Esto se debe a que tom el siguiente camino seg n los pesos de
55           las aristas:")
56     print(f"        {ruta_str}")

```

Listing 8: Tabla de costos y resumen explicativo del camino

6. Referencias

- García, J. (2022). *Algoritmo de Dijkstra explicado con ejemplos*. Recuperado de <https://www.freecodecamp.org/espanol/news/algoritmo-de-dijkstra-explicado-con-ejemplos/>
- Rojas, M. (2020). *Qué es el algoritmo de Dijkstra y cómo funciona*. Recuperado de <https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-graph-data-structure/>
- Fernández, C. (2021). *Grafos dirigidos y ponderados: conceptos y aplicaciones*. Recuperado de <https://es.khanacademy.org/computing/computer-science/algorithms/graph-representation/a/representing-graphs>
- Universidad Autónoma de México. (2021). *Algoritmos de rutas mínimas en grafos*. Recuperado de <https://blog.mat.uam.es/algoritmos-de-grafos-rutas-minimas/>
- Sánchez, M. (2016, septiembre 9). *Algoritmo de Dijkstra explicado con matriz de adyacencia* [Video]. YouTube. <https://www.youtube.com/watch?v=4I7W5WUQQI>